UỶ BAN NHÂN DÂN THÀNH PHỐ
HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO LAB04 – THỎA MÃN RÀNG BUỘC

MÔN: TRÍ TUỆ NHÂN TẠO NÂNG CAO

Giảng viên hướng dẫn: Đỗ Như Tài

Sinh viên thực hiện:
Huỳnh Tấn Phi Hùng - 3124720026
Lê Tấn Phát - 3124720050
Lê Ngọc Phúc – 3124720053
Đỗ Xuân Triệu - 3124720072

TP. HỒ CHÍ MINH, ngày 23 tháng 10 năm 2026

MỤC LỤC

1. Giới thiệu và môi trường thực nghiệm 1
1.1. Môi trường và cách chạy lại 1
1.2. Quy ước các đại lượng 1
2. Bài 1-2: Biểu diễn CSP, kiểm tra đầy đủ và nhất quán 1
2.1. Sinh đồ thị và mô hình CSP 1
2.2. Phép gán đầy đủ, hợp lệ và nghiệm 2
2.3. Kiểm thử complete() và consistent() 2
3. Bài 3: Backtracking, MRV và LCV 3
3.1. Backtracking cơ bản 3
3.2. MRV và LCV 3
4. Bài 4: Forward Checking, Min-Conflicts và thực nghiệm 4
4.1. Forward Checking 4
4.2. Min-Conflicts 4
4.3. Thiết lập thực nghiệm 4
4.4. Kết quả với n = 10 (100 đồ thị, 3 và 4 màu) 4
4.5. Tăng kích thước n (4 màu) 5
4.6. Độ ổn định theo seed 6
4.7. Min-Conflicts 6
5. Bài 5: Biểu diễn bài toán n quân hậu 8
6. Bài 6: Các biến thể Hill Climbing 9
6.1. Ba biến thể 9
6.2. Mắc kẹt: cực trị cục bộ và plateau 10
7. Bài 7: Random Restart và Simulated Annealing 10
7.1. Random Restart (Task 4) 10
7.2. Simulated Annealing (Task 5) 11
8. Bài 8: Đánh giá tổng hợp và bài nâng cao 12
8.1. So sánh các thuật toán (n = 4 và n = 8, 100 lượt) 12
8.2. Hội tụ 12
8.3. Khả năng mở rộng theo n 13
8.4. Bài nâng cao: các toán tử di chuyển cục bộ khác 14
9. Bảng ghi kết quả thí nghiệm (bảng 4.2) 15
10. Nhận xét tổng hợp 16
11. Trả lời câu hỏi thảo luận 16
12. Kết luận 18
Bảng phân công: 18

1. Giới thiệu và môi trường thực nghiệm

Báo cáo trình bày kết quả Lab04 về bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem - CSP) gồm hai phần. Phần thứ nhất là tô màu đồ thị phẳng (đồ thị Delaunay của các điểm ngẫu nhiên) với 3 và 4 màu, giải bằng backtracking có các cải tiến MRV, LCV, Forward Checking và bằng tìm kiếm cục bộ Min-Conflicts. Phần thứ hai là bài toán n quân hậu giải bằng các biến thể hill climbing, random restart và simulated annealing. Toàn bộ mã nguồn, kết quả chạy và giải thích nằm trong hai notebook CSP_graph_coloring_example.ipynb và n_queens.ipynb nộp kèm; báo cáo này tổng hợp các số liệu từ lần chạy cuối cùng của hai notebook đó.

1.1. Môi trường và cách chạy lại

Python 3.13.16 (Linux x86_64) với numpy 2.5.3, scipy 1.18.1, matplotlib 3.11.2, pandas 3.0.5. Chỉ dùng các thư viện này; không dùng thư viện cài sẵn thuật toán tìm kiếm.

Notebook không phụ thuộc đường dẫn cá nhân; chạy lại từ đầu bằng "Restart & Run All". Ô đầu tiên của mỗi notebook in phiên bản thư viện.

Mọi thí nghiệm đặt seed cố định và các thuật toán được so sánh trên cùng tập đồ thị / cùng bàn cờ ban đầu, vì vậy COUNT, số bước, số lần đánh giá và tỉ lệ thành công lặp lại được. Riêng thời gian chạy phụ thuộc máy nên chỉ nên so sánh tương đối.

Thời gian đo bằng time.perf_counter(). COUNT ở bài tô màu là số nút backtracking được kiểm tra (số lần gọi đệ quy), không đồng nhất với số lần đánh giá hàm mục tiêu ở bài n quân hậu.

1.2. Quy ước các đại lượng

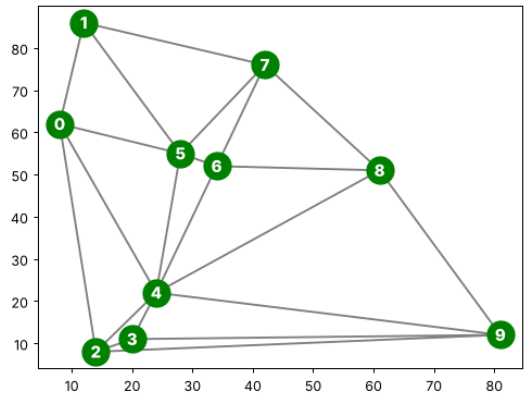
Đại lượng	Định nghĩa
COUNT	Số lần gọi đệ quy của backtracking (số nút được xét), cùng định nghĩa với biến COUNT của notebook gốc.
Hết giới hạn	Backtracking dừng vì vượt trần 200 000 nút; chưa kết luận có nghiệm hay không.
Chứng minh vô nghiệm	Backtracking duyệt hết không gian tìm kiếm mà không tìm được nghiệm (kết luận chắc chắn).
Nước đi cải thiện (uphill)	Bước của Min-Conflicts làm giảm số xung đột của đỉnh được sửa.
h (hoặc f)	Số cặp quân hậu xung đột của một bàn cờ; h = 0 là lời giải.
Thành công	Lượt chạy kết thúc bằng một phép gán đầy đủ và hợp lệ (tô màu) hoặc h = 0 (n quân hậu).

Bảng 1. Các đại lượng dùng trong báo cáo

2. Bài 1-2: Biểu diễn CSP, kiểm tra đầy đủ và nhất quán

2.1. Sinh đồ thị và mô hình CSP

Ta đặt n = 10 điểm ngẫu nhiên trong $[0, 100)^2$ (seed 1111) rồi tam giác hóa Delaunay; hai điểm cùng thuộc một tam giác thì được nối bằng một cạnh (Hình 1). Danh sách láng giềng của mỗi điểm được lấy từ vertex_neighbor_vertices của scipy.



Hình 1. Đồ thị Delaunay với n = 10 điểm (seed 1111): 21 cạnh

CSP được mô hình hóa bởi bộ ba (X, D, C):

X = {0, 1, …, n-1}: mỗi đỉnh là một biến (tên dạng chuỗi);

D: miền của mỗi biến là tập màu, D = {red, blue, green, orange} (4 màu) hoặc {red, blue, green} (3 màu);

C: với mỗi cạnh (i, j) của đồ thị, color(i) ≠ color(j).

Vì sao chỉ tạo ràng buộc cho i < j. Ràng buộc "khác màu" là đối xứng, còn danh sách láng giềng của Delaunay liệt kê mỗi cạnh hai lần (một lần ở mỗi đầu). Chỉ giữ cạnh với i < j nên mỗi ràng buộc được lưu và kiểm tra đúng một lần, tránh nhân đôi chi phí mà không thêm thông tin. Với seed 1111, đồ thị có |X| = 10, |C| = 21 và bậc trung bình 4,2; số cạnh tối đa của đồ thị phẳng tam giác hóa là 3n - 6 = 24.

2.2. Phép gán đầy đủ, hợp lệ và nghiệm

Phép gán đầy đủ (complete) gán giá trị cho mọi biến; phép gán nhất quán (consistent) không vi phạm ràng buộc nào giữa các biến đã gán. Hai tính chất độc lập: phép gán rỗng nhất quán nhưng chưa đầy đủ; tô mọi đỉnh cùng màu thì đầy đủ nhưng không nhất quán. Nghiệm của CSP là phép gán vừa đầy đủ vừa nhất quán, vì vậy một phép gán đầy đủ chưa chắc là nghiệm. Backtracking duy trì phép gán một phần nhất quán và mở rộng dần; Min-Conflicts duy trì phép gán đầy đủ và sửa dần các vi phạm. Cấu trúc lưu trữ csp là từ điển gồm variables (danh sách biến), domains (từ điển biến → danh sách màu) và constraints (tập các cặp biến không được cùng màu).

2.3. Kiểm thử complete() và consistent()

Năm ca kiểm thử dưới đây có kết quả mong đợi biết trước; trong notebook, assert bảo đảm kết quả thực tế khớp mong đợi (cột "thực tế" trùng cột "mong đợi" ở mọi ca).

Ca kiểm thử	complete (mong đợi)	consistent (mong đợi)	Thực tế khớp?
T1: phép gán rỗng {}	False	True	Có
T2: một phần, hai đỉnh kề khác màu	False	True	Có
T3: một phần, hai đỉnh kề cùng màu	False	False	Có
T4: đầy đủ nhưng mọi đỉnh cùng 'red'	True	False	Có
T5: đầy đủ và hợp lệ (tô tham lam)	True	True	Có

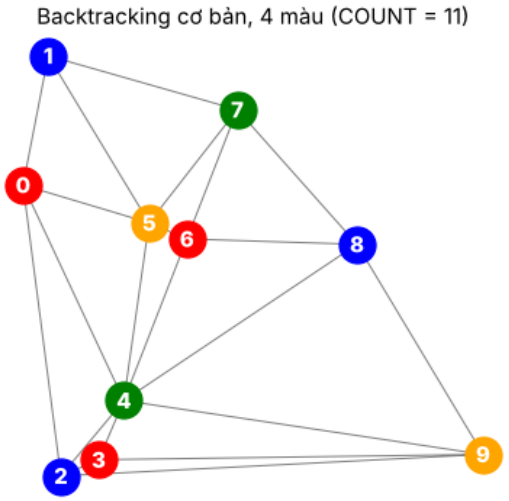
Bảng 2. Kết quả kiểm thử complete() và consistent()

Ca T4 cho thấy complete() không đồng nghĩa consistent(): một phép gán đầy đủ vẫn có thể vi phạm ràng buộc. Ca T1 ngược lại: phép gán rỗng nhất quán một cách tầm thường nhưng không đầy đủ.

3. Bài 3: Backtracking, MRV và LCV

3.1. Backtracking cơ bản

Phiên bản gốc chọn biến chưa gán đầu tiên (0, 1, 2, …) và thử các màu theo thứ tự trong miền. Trên đồ thị gốc (4 màu) thuật toán tìm được nghiệm sau COUNT = 11 nút (n + 1, tức không phải quay lui lần nào) trong khoảng vài phần mười mili giây. Nghiệm hợp lệ và đầy đủ được trực quan hóa ở Hình 2. Mã vẽ của notebook gốc giả định thứ tự các giá trị trong từ điển kết quả trùng thứ tự biến và gây lỗi khi kết quả là None hoặc khi dùng MRV; ta đã sửa bằng cách lấy màu theo tên biến.



Hình 2. Nghiệm 4 màu do backtracking cơ bản tìm được (COUNT = 11)

3.2. MRV và LCV

MRV (Minimum Remaining Values): chọn biến chưa gán có ít giá trị hợp lệ nhất (giá trị không trùng màu láng giềng đã gán); biến ít lựa chọn nhất dễ thất bại nhất nên cắt nhánh sớm.

LCV (Least Constraining Value): với biến đã chọn, thử trước màu làm các láng giềng chưa gán mất ít lựa chọn nhất.

Hàm solve_basic giữ nguyên định nghĩa COUNT của bản gốc và thêm tham số mrv, lcv. Khi tắt cả hai, COUNT và nghiệm trùng khớp với backtrack() gốc (kiểm tra trong notebook), nên so sánh là công bằng. Tham số limit đặt trần số nút để phân biệt "hết giới hạn" với "chứng minh vô nghiệm".

Phiên bản	n = 10: COUNT	n = 10: thời gian (ms)	n = 60: COUNT	n = 60: thời gian (ms)
Gốc	11	0,03	107	0,28
MRV	11	0,07	240	3,12
LCV	11	0,07	529	3,16
MRV + LCV	11	0,09	80	1,82

Bảng 3. So sánh MRV/LCV với bản gốc trên một đồ thị (n = 10, seed 1111; n = 60, seed 42; 4 màu)

Trên đồ thị gốc n = 10 cả bốn cấu hình đều có COUNT = 11 nên chưa phân biệt được. Trên một đồ thị n = 60, MRV riêng lẻ (240 nút) và LCV riêng lẻ (529 nút) thậm chí tệ hơn bản gốc (107 nút); chỉ MRV + LCV (80 nút) tốt hơn. Điều này cho thấy heuristic không bảo đảm thắng trên từng ví dụ, và mỗi nút của MRV/LCV tốn thêm thời gian vì phải tính lại số giá trị hợp lệ. Vì vậy phần 4 so sánh trên nhiều đồ thị rồi lấy trung bình.

4. Bài 4: Forward Checking, Min-Conflicts và thực nghiệm

4.1. Forward Checking

Sau mỗi phép gán var = val, thuật toán loại val khỏi miền của các láng giềng chưa gán; nếu một miền rỗng thì thất bại ngay mà không cần đệ quy tiếp. Khi quay lui phải hoàn tác đúng các miền đã sửa (hàm undo_forward_check). Cài đặt dùng bản sao miền thu hẹp dần và có thể kết hợp với MRV, LCV (khi đó các heuristic làm việc trên miền đã thu hẹp). Cả ba cấu hình Forward Checking trên đồ thị gốc đều trả về nghiệm hợp lệ, đầy đủ.

4.2. Min-Conflicts

Min-Conflicts là hill climbing cục bộ trên phép gán đầy đủ: (1) khởi tạo mỗi đỉnh một màu ngẫu nhiên; (2) mỗi bước chọn ngẫu nhiên một đỉnh đang vi phạm rồi đổi sang màu ít xung đột nhất với láng giềng (hòa thì chọn ngẫu nhiên); (3) dừng khi không còn xung đột; nếu sau max_steps = 10n bước chưa xong thì khởi động lại ngẫu nhiên, tối đa 10 lần. Ta ghi tổng số bước, số nước đi cải thiện (làm giảm số xung đột của đỉnh) và số lần khởi động lại.

4.3. Thiết lập thực nghiệm

Với mỗi n, ta sinh nhiều đồ thị ngẫu nhiên (seed khác nhau) và chạy mọi thuật toán trên cùng tập đồ thị. Trần backtracking là 200 000 nút. Mỗi thuật toán báo cáo: tỉ lệ tìm được nghiệm, tỉ lệ chứng minh vô nghiệm, tỉ lệ hết giới hạn, COUNT trung bình và thời gian trung bình. Mọi nghiệm tìm được đều được kiểm tra lại bằng complete() và consistent() bằng assert.

4.4. Kết quả với n = 10 (100 đồ thị, 3 và 4 màu)

Số cạnh trung bình của các đồ thị n = 10 là 21,05 (tối đa 3n - 6 = 24).

Thuật toán	Màu	Có nghiệm (%)	Chứng minh vô nghiệm (%)	Hết giới hạn (%)	COUNT TB	Thời gian TB (ms)
Backtracking	4	100	0	0	11,59	0,02
MRV + Backtracking	4	100	0	0	11,30	0,06
Forward Checking	4	100	0	0	11,20	0,02
MRV + Forward Checking	4	100	0	0	11,18	0,05
MRV + LCV + Forward Checking	4	100	0	0	11,17	0,12

Thuật toán	Màu	Có nghiệm (%)	Chứng minh vô nghiệm (%)	Hết giới hạn (%)	COUNT TB	Thời gian TB (ms)
Backtracking	3	5	95	0	47,27	0,14
MRV + Backtracking	3	5	95	0	32,01	0,29
Forward Checking	3	5	95	0	29,92	0,09
MRV + Forward Checking	3	5	95	0	26,31	0,22
MRV + LCV + Forward Checking	3	5	95	0	26,31	0,27

Bảng 4. Backtracking và các cải tiến trên 100 đồ thị n = 10

Với 4 màu mọi thuật toán giải 100% với COUNT gần bằng 11, nên heuristic gần như không có tác dụng ở n nhỏ. Với 3 màu, chỉ 5% đồ thị có nghiệm; 95% được backtracking chứng minh vô nghiệm (không phải hết giới hạn). Lý do: đồ thị Delaunay là tam giác hóa, và một tam giác hóa phẳng chỉ tô được bằng 3 màu khi mọi đỉnh trong đều có bậc chẵn, điều rất hiếm với điểm ngẫu nhiên. Ở các bài toán vô nghiệm này heuristic phát huy tác dụng: COUNT giảm từ 47,27 (Backtracking) xuống 32,01 (MRV), 29,92 (FC) và 26,31 (MRV + FC), tức khoảng 44% số nút.

4.5. Tăng kích thước n (4 màu)

Với n ∈ {10, 20, 50, 100, 200}, mỗi cỡ dùng 20 đồ thị. Số cạnh trung bình tăng gần tuyến tính theo n và luôn dưới 3n - 6; tỉ lệ cạnh/đỉnh tiến dần đến 3 (Bảng 5).

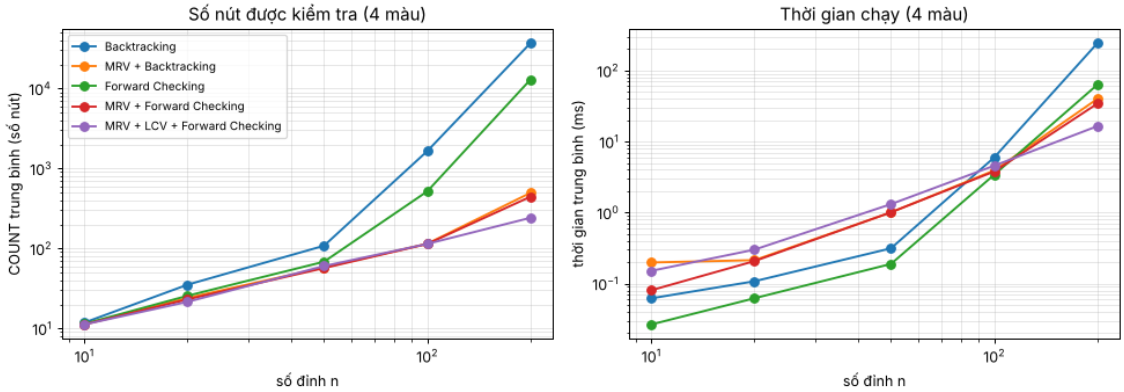
n	Số cạnh TB	Cạnh / n	Giới hạn 3n - 6
10	21,4	2,14	24
20	49,8	2,49	54
50	137,1	2,74	144
100	284,3	2,84	294
200	582,6	2,91	594

Bảng 5. Số cạnh trung bình của đồ thị Delaunay theo n

Thuật toán	n	Có nghiệm (%)	Hết giới hạn (%)	COUNT TB	Thời gian TB (ms)
Backtracking	10	100	0	12	0,06
MRV + Backtracking	10	100	0	11	0,20
Forward Checking	10	100	0	11	0,03
MRV + Forward Checking	10	100	0	11	0,08
MRV + LCV + Forward Checking	10	100	0	11	0,15
Backtracking	50	100	0	108	0,31
MRV + Backtracking	50	100	0	58	1,01
Forward Checking	50	100	0	68	0,19
MRV + Forward Checking	50	100	0	57	1,01
MRV + LCV + Forward Checking	50	100	0	60	1,31
Backtracking	100	100	0	1 649	5,97
MRV + Backtracking	100	100	0	115	3,91
Forward Checking	100	100	0	517	3,41

Thuật toán	n	Có nghiệm (%)	Hết giới hạn (%)	COUNT TB	Thời gian TB (ms)
MRV + Forward Checking	100	100	0	114	3,78
MRV + LCV + Forward Checking	100	100	0	114	4,57
Backtracking	200	90	10	37 027	243,41
MRV + Backtracking	200	100	0	496	39,98
Forward Checking	200	95	5	12 952	63,36
MRV + Forward Checking	200	100	0	444	34,27
MRV + LCV + Forward Checking	200	100	0	242	16,47

Bảng 6. Hiệu năng backtracking theo n (4 màu, 20 đồ thị mỗi cỡ; một phần các cỡ n)



Hình 3. COUNT và thời gian chạy theo n (log-log, 4 màu)

Nhận xét: Backtracking cơ bản bùng nổ: COUNT trung bình tăng từ 11,8 (n = 10) lên 1 649 (n = 100) và 37 027 (n = 200), và 10% đồ thị n = 200 vượt trần 200 000 nút. Forward Checking đơn thuần giảm mạnh (517 nút ở n = 100) nhưng vẫn có 5% hết giới hạn ở n = 200. MRV là heuristic hiệu quả nhất: MRV + Backtracking chỉ 496 nút ở n = 200 và giải 100%; MRV + LCV + FC chỉ 242 nút. Tuy nhiên ở n nhỏ (≤ 50) các heuristic có COUNT thấp hơn nhưng thời gian lại cao hơn Backtracking trơn, vì chi phí tính heuristic trên mỗi nút; chúng chỉ có lợi về thời gian khi bài toán đủ lớn.

4.6. Độ ổn định theo seed

Lặp lại thí nghiệm n = 100 trên 5 nhóm seed khác nhau (mỗi nhóm 20 đồ thị). MRV + Forward Checking luôn giải 100% với COUNT trung bình 121,3 và độ lệch chuẩn nhỏ; Backtracking cơ bản có COUNT trung bình khoảng 3 640 nhưng độ lệch chuẩn khoảng 4 839, tức biến thiên rất lớn: phân bố có đuôi nặng, vài đồ thị khó kéo trung bình lên. Vì vậy cần đánh giá trên nhiều đồ thị thay vì một ví dụ.

4.7. Min-Conflicts

n	Màu	Có nghiệm (%)	Số bước TB	Nước đi cải thiện TB	Khởi động lại TB	Thời gian TB (ms)
10	4	100	12	3,9	0,00	0,2
20	4	100	49	10,3	0,00	1,1
50	4	100	375	35,5	0,45	13,9
100	4	100	1 338	98,0	0,90	95,9
200	4	75	9 591	545,2	4,40	1150,2

n	Màu	Có nghiệm (%)	Số bước TB	Nước đi cải thiện TB	Khởi động lại TB	Thời gian TB (ms)
10	3	5	950	39,6	9,50	11,5
20	3	0	2 000	91,5	10,00	33,0
50	3	0	5 000	230,3	10,00	169,1
100	3	0	10 000	474,5	10,00	573,4
200	3	0	20 000	963,4	10,00	2135,9

Bảng 7. Min-Conflicts theo n (20 đồ thị mỗi cỡ; max_steps = 10n mỗi lần, tối đa 10 lần khởi động lại)

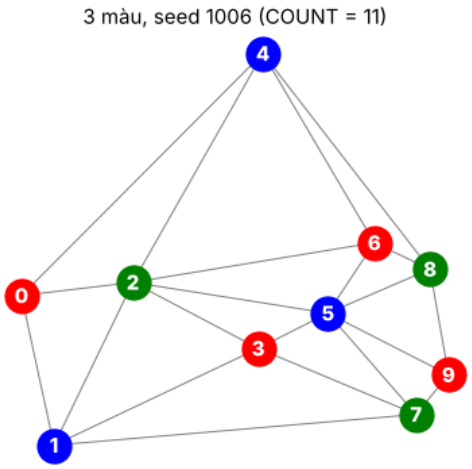
Với 4 màu, Min-Conflicts giải 100% đến n = 100 nhưng chỉ còn 75% ở n = 200 (ngân sách 10n bước × 10 lần). Số bước tăng nhanh (12 → 1 338 → 9 591 khi n = 10 → 100 → 200) và thời gian ở n = 200 (khoảng 1,2 s) chậm hơn MRV + LCV + FC (khoảng 15-20 ms) nhiều lần. Số nước đi cải thiện chỉ là một phần nhỏ trong tổng số bước vì nhiều bước là đi ngang trên plateau.

Với 3 màu, Min-Conflicts gần như luôn thất bại (5% ở n = 10, 0% từ n = 20). Điều này không phải điểm yếu riêng của thuật toán: phần lớn đồ thị 3 màu vốn vô nghiệm (xem 4.4), mà Min-Conflicts không thể chứng minh vô nghiệm, nó chỉ dùng hết ngân sách rồi dừng. Để đánh giá công bằng, ta lọc ra 20 đồ thị n = 10 có nghiệm 3 màu (do backtracking đã chứng minh) và chạy lại:

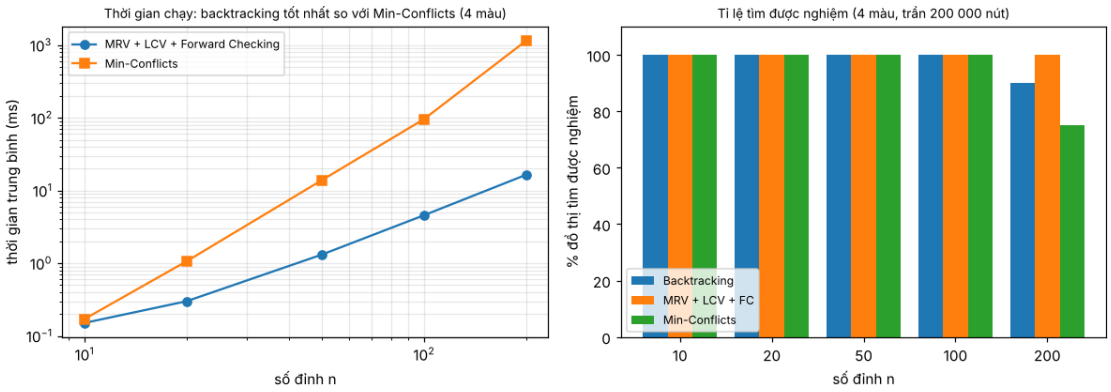
Cấu hình	Số đồ thị	Có nghiệm (%)	Số bước TB	Nước đi cải thiện TB	Khởi động lại TB
3 màu, max_restarts = 20	20	100	25,45	4,8	0,00
3 màu, max_restarts = 100	20	100	25,45	4,8	0,00

Bảng 8. Min-Conflicts trên các đồ thị n = 10 có nghiệm 3 màu

Khi bài toán có nghiệm, Min-Conflicts giải 100% với trung bình 25,45 bước, không cần khởi động lại. Đồ thị gốc (seed 1111) với 3 màu là một ví dụ vô nghiệm: backtracking chứng minh điều đó chỉ sau 28 nút. Hình 4 minh họa một đồ thị n = 10 có nghiệm 3 màu.



Hình 4. Nghiệm 3 màu của một đồ thị n = 10 có nghiệm (seed 1006, COUNT = 11)



Hình 5. So sánh MRV + LCV + Forward Checking với Min-Conflicts: thời gian (trái) và tỉ lệ tìm được nghiệm (phải)

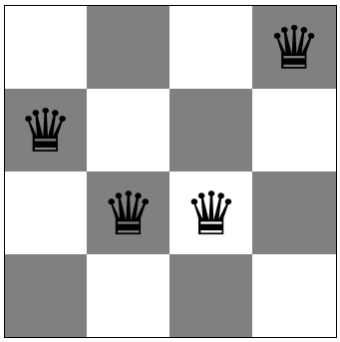
Hình 5 tóm tắt khác biệt về bản chất: backtracking có đảm bảo (đầy đủ, chứng minh được vô nghiệm) và với heuristic tốt vẫn nhanh ở n = 200; Min-Conflicts nhẹ về bộ nhớ nhưng không có đảm bảo và phải tăng ngân sách bước khi n lớn.

5. Bài 5: Biểu diễn bài toán n quân hậu

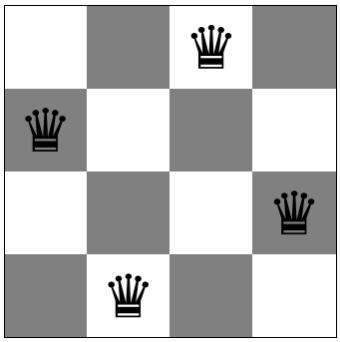
Mỗi cột có đúng một quân hậu; board[i] là chỉ số hàng của quân hậu ở cột i. Cách biểu diễn này loại sẵn mọi xung đột cùng cột. Hàm mục tiêu cần cực tiểu hóa là số cặp quân hậu cùng hàng hoặc cùng đường chéo:

h(board) = #{ (i, j): i < j và (board[i] = board[j] hoặc |board[i] - board[j]| = |i - j|) }.

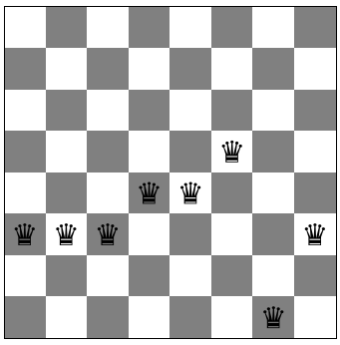
Hàm conflicts(board) tính đúng đại lượng này trong O(n) bằng cách đếm số quân hậu trên từng hàng và từng đường chéo rồi cộng C(k, 2) cho mỗi nhóm có k quân. Notebook kiểm tra: (1) conflicts trùng với đếm vét cạn theo định nghĩa trên 1 500 bàn cờ ngẫu nhiên (n = 4…12); (2) mệnh đề h = 0 khi và chỉ khi không có cặp quân hậu nào tấn công nhau bằng cách duyệt toàn bộ $4^4 = 256$ bàn cờ n = 4 (có đúng 2 bàn cờ h = 0) và $6^6 = 46\,656$ bàn cờ n = 6 (đúng 4 bàn cờ h = 0), khớp số lời giải đã biết của các bài toán 4 và 6 quân hậu.



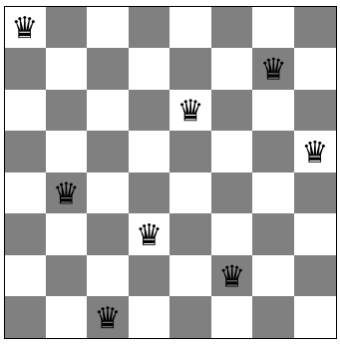
Hình 6a. n = 4, ngẫu nhiên: h = 3



Hình 6b. n = 4, lời giải [1, 3, 0, 2]: h = 0



Hình 6c. n = 8, ngẫu nhiên: h = 11



Hình 6d. n = 8, lời giải [0, 4, 7, 5, 2, 6, 1, 3]: h = 0

6. Bài 6: Các biến thể Hill Climbing

6.1. Ba biến thể

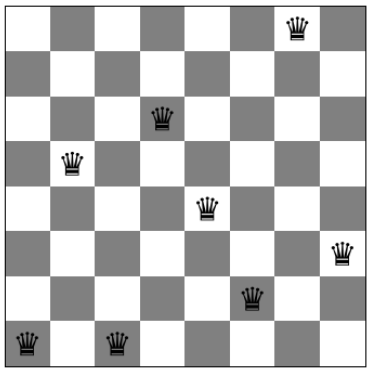
Task 1 - Steepest-ascent: đánh giá toàn bộ n(n - 1) nước đi (đổi hàng của một quân hậu trong cột của nó), chọn nước cho h nhỏ nhất (hòa thì chọn ngẫu nhiên); dừng khi h = 0 hoặc không còn nước nào cải thiện.

Task 2 - Inefficient Stochastic HC: liệt kê mọi nước đi cải thiện chặt rồi chọn ngẫu nhiên một nước; dừng khi tập này rỗng.

Task 3 - Efficient Stochastic (first-choice) HC: mỗi lần sinh một lân cận ngẫu nhiên và nhận nếu h giảm ngặt; dừng khi h = 0 hoặc sau x lần thử liên tiếp không cải thiện.

Điều kiện dừng của Task 3: Gọi N = n(n - 1) là số lân cận. Lấy mẫu có hoàn lại, sau N(ln N + c) lần thử xác suất đã thử mọi lân cận xấp xỉ exp(-e^(-c)) (bài toán coupon collector); chọn c = 2 (khoảng 87%), tức x = ⌈N(ln N + 2)⌉, bằng 54 với n = 4 và 338 với n = 8. Nhờ có ngưỡng x thuật toán chắc chắn dừng, không lặp vô hạn; cái giá là cực trị cục bộ chỉ được phát hiện theo xác suất.

Mỗi lần chạy trả về bàn cờ cuối, h cuối, số bước (nước đi được nhận), số lần đánh giá conflicts() và lịch sử h. Ví dụ steepest-ascent trên bàn cờ 8×8 (seed 1234): từ [7, 3, 6, 5, 4, 4, 0, 1] (h = 9) sau 4 bước, 280 lần đánh giá dừng ở [7, 3, 7, 2, 4, 6, 0, 5] với h = 1 - mắc kẹt (Hình 7).



Hình 7. Steepest-ascent trên bàn cờ 8×8 mắc kẹt với h = 1

6.2. Mắc kẹt: cực trị cục bộ và plateau

Ta chạy 100 bàn cờ ngẫu nhiên n = 8 cho mỗi biến thể (cùng seed) và với các lần kết thúc ở h > 0 phân loại trạng thái dừng: cực trị chặt (không lân cận nào có h ≤ h hiện tại) hoặc plateau (còn lân cận có h bằng h hiện tại nhưng thuật toán chỉ nhận nước cải thiện chặt).

Thuật toán	Kết thúc h = 0 (%)	h cuối TB	Số bước TB	Số lần đánh giá TB	Mắc kẹt	Cực trị chặt	Plateau
Steepest-ascent	14	1,29	3,29	232,4	86	0	86
Stochastic HC (mọi nước đi)	14	1,32	4,82	318,1	86	3	83
First-choice HC	11	1,36	4,97	374,3	89	2	87

Bảng 9. Ghi nhận mắc kẹt của ba biến thể hill climbing (n = 8, 100 bàn cờ)

Chỉ khoảng 11-14% lượt chạy đạt h = 0. Hầu hết các trạng thái kẹt không phải cực trị chặt mà là plateau: ở h nhỏ (thường h = 1) luôn còn nước đi ngang nhưng thuật toán chỉ nhận nước cải thiện chặt nên dừng. Số lần đánh giá của steepest-ascent (khoảng 232) thấp hơn first-choice (khoảng 374) trên n = 8 vì steepest chỉ cần vài bước, mỗi bước quét 56 lân cận, còn first-choice phải chờ đủ x = 338 lần thử liên tiếp không cải thiện mới dừng.

7. Bài 7: Random Restart và Simulated Annealing

7.1. Random Restart (Task 4)

Mỗi biến thể được chạy lại tối đa 100 lần từ các bàn cờ ngẫu nhiên mới, dừng sớm khi h = 0; mỗi dòng dưới đây gồm 100 phép thử độc lập (seed 2024).

Thuật toán + restart	n	Giải được trong ≤ 100 lần (%)	TB số lần khởi động	Max số lần khởi động
Steepest-ascent	4	100	2,32	15
Steepest-ascent	8	100	7,90	28
Stochastic HC (mọi nước đi)	4	100	2,89	14
Stochastic HC (mọi nước đi)	8	100	6,94	45
First-choice HC	4	100	2,65	12
First-choice HC	8	100	8,69	51

Bảng 10. Hill climbing với random restart

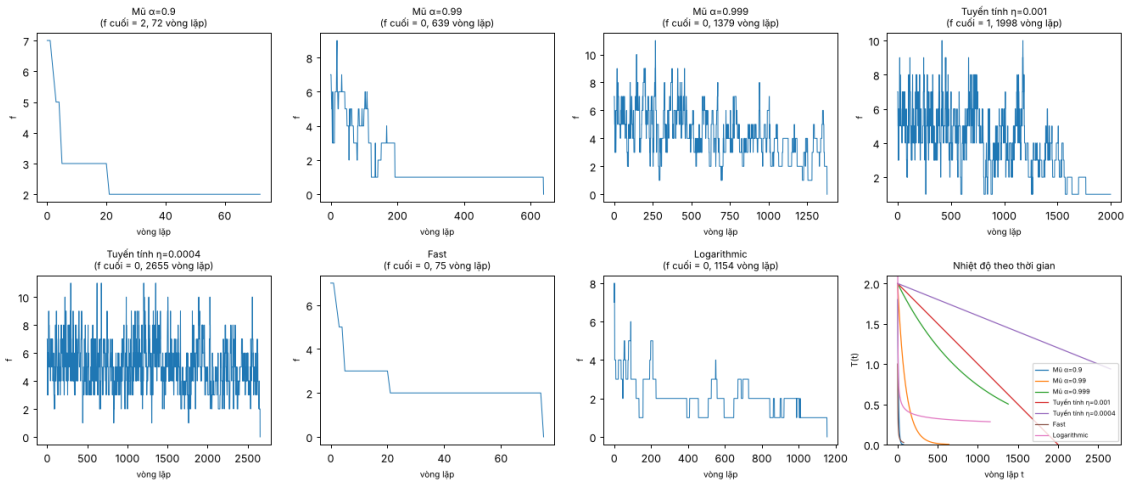
Cả ba biến thể đều đạt 100% thành công. Nếu một lần chạy đơn thành công với xác suất p thì số lần khởi động kỳ vọng khoảng 1/p; với n = 8, p chỉ khoảng 8-19% nên cần trung bình 7-9 lần, và lần xấu nhất lên đến 51 lần. Restart tăng cơ hội tìm nghiệm nhưng cũng nhân chi phí tổng (xem số lần đánh giá ở Bảng 12).

7.2. Simulated Annealing (Task 5)

Ở mỗi vòng lặp t, SA sinh một lân cận ngẫu nhiên, tính Δh = h(mới) - h(hiện tại) và chấp nhận nếu Δh < 0, ngược lại chấp nhận với xác suất p = exp(-Δh/T(t)); nhiệt độ giảm theo lịch T(t). T cao thì khám phá, T thấp thì giống hill climbing. SA dừng khi h = 0, khi T < 10^{-3} hoặc sau 5 000 vòng; kết quả là trạng thái tốt nhất từng gặp, và thành công nghĩa là h = 0 (không nhầm với "h nhỏ nhất"). Ta thử bảy lịch với T_0 = 2:

Lịch nhiệt độ	Thành công (%)	h cuối TB	Số vòng lặp TB	Thời gian TB (ms)
Mũ α=0.9	7	1,63	70,9	0,99
Mũ α=0.99	60	0,40	529,7	8,04
Mũ α=0.999	98	0,02	1717,3	23,12
Tuyến tính η=0.001	67	0,33	1684,0	23,25
Tuyến tính η=0.0004	93	0,07	3296,3	46,83
Fast	88	0,12	732,8	11,89
Logarithmic	100	0,00	712,1	12,44

Bảng 11. So sánh các lịch giảm nhiệt (n = 8, 100 lần chạy mỗi lịch)



Hình 8. Đường h theo vòng lặp (trên) và đường nhiệt độ T(t) tương ứng (dưới) cho từng lịch

Giảm nhiệt quá nhanh (mũ α = 0,9) làm SA "đóng băng" sau vài chục vòng nên hành xử như hill climbing (7% thành công). Giảm chậm hơn (α = 0,999, tuyến tính η nhỏ) cho phép khám phá lâu và nâng tỉ lệ thành công lên 93-98%, với cái giá là số vòng lặp tăng. Lịch logarithmic cho 100% thành công ở n = 8 (ta chỉ khảo sát các lịch ở n = 8). Lịch mũ α = 0,999 được chọn làm mặc định cho các so sánh ở Bài 8.

8. Bài 8: Đánh giá tổng hợp và bài nâng cao

8.1. So sánh các thuật toán (n = 4 và n = 8, 100 lượt)

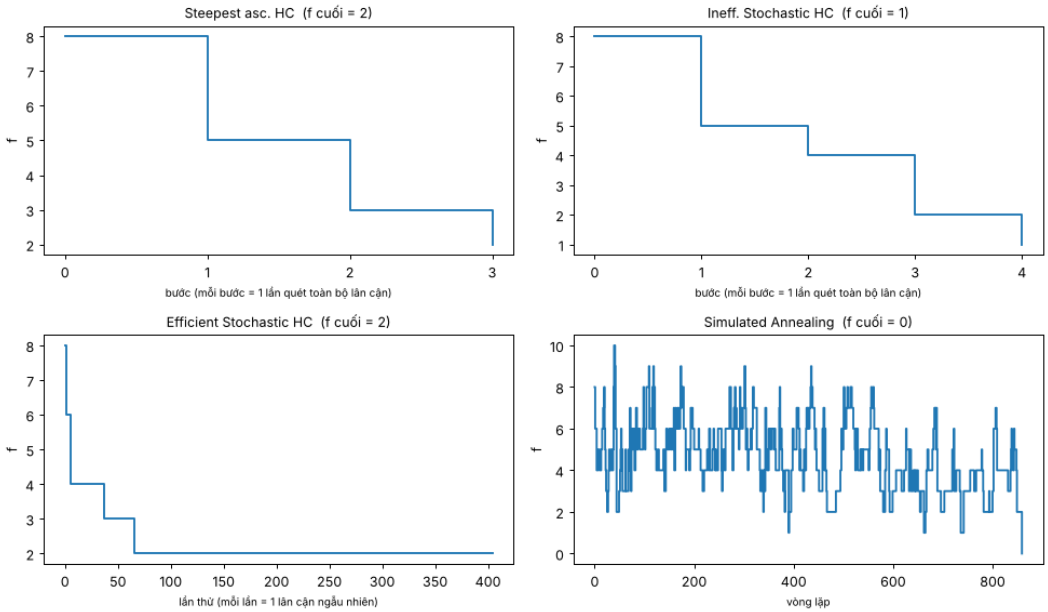
Mỗi dòng là trung bình của 100 lượt chạy từ 100 bàn cờ khởi tạo khác nhau (seed cố định, mọi thuật toán dùng cùng chuỗi bàn cờ). "Số bước" là số nước đi được nhận, "số lần đánh giá" là số lần gọi conflicts().

Thuật toán	n	Thời gian TB (ms)	h cuối TB	Thành công (%)	Số bước TB	Số lần đánh giá TB
Steepest asc. HC	4	0,094	0,57	47	1,5	24
Ineff. Stochastic HC	4	0,108	0,89	29	1,8	30
Efficient Stochastic HC	4	0,421	0,70	36	1,8	46
Simulated Annealing	4	0,972	0,00	100	72,5	98
Steepest asc. + Restart	4	0,274	0,00	100	4,1	67
Steepest asc. HC	8	1,404	1,20	19	3,3	232
Ineff. Stochastic HC	8	2,211	1,34	11	5,1	334
Efficient Stochastic HC	8	4,762	1,44	8	4,8	374
Simulated Annealing	8	22,392	0,01	99	655,9	1641
Steepest asc. + Restart	8	12,200	0,00	100	24,7	1763

Bảng 12. Bảng so sánh thuật toán trên bài toán n quân hậu (100 lượt mỗi dòng)

Nhận xét: (i) Trên n = 8 ba thuật toán leo đồi một lần chạy chỉ giải 8-19%; phần còn lại kẹt ở cực trị cục bộ hoặc plateau. (ii) SA giải 99% (100% với n = 4) nhưng đắt: khoảng 22 ms và hơn 1 600 lần đánh giá. (iii) Steepest-ascent + random restart đạt 100% với khoảng 12 ms nên rẻ hơn SA trên bài toán nhỏ này. (iv) Với n = 4 chỉ có 256 trạng thái nên mọi thuật toán đều rất nhanh; khác biệt chủ yếu là tỉ lệ thành công. (v) Các giá trị h cuối trung bình của leo đồi (khoảng 1,2-1,4 ở n = 8) cho thấy thuật toán thường dừng rất gần nghiệm nhưng không tới được.

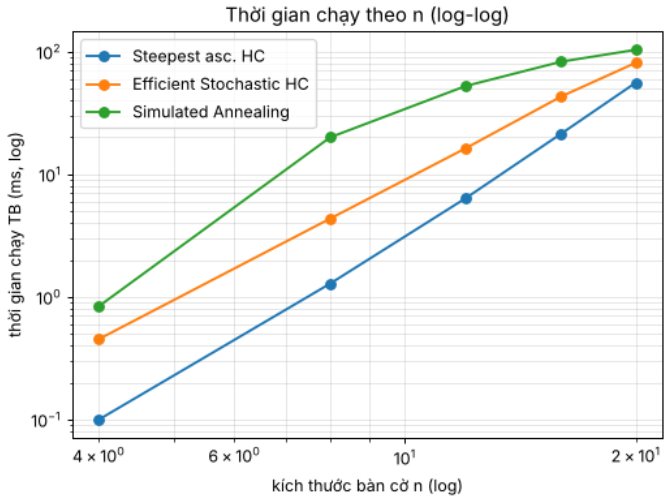
8.2. Hội tụ



Hình 9. Đường hội tụ của một lượt chạy đại diện của từng thuật toán trên 8 quân hậu

Steepest-ascent và stochastic HC giảm h rất nhanh ở những bước đầu (mỗi bước gỡ nhiều xung đột) rồi đi ngang thành bậc thang phẳng ở h > 0 khi kẹt. First-choice có dạng bậc thang dài vì phần lớn các lần thử bị từ chối và dễ kẹt ở plateau. SA dao động mạnh khi nhiệt độ còn cao rồi hạ dần và dừng khi h = 0. Tỉ lệ lượt chạy kết thúc ở cực trị cục bộ trên n = 8 là 81% (steepest), 89% (stochastic), 92% (first-choice) và chỉ 1% (SA): ba thuật toán leo đồi hay mắc kẹt nhất, SA hầu như không.

8.3. Khả năng mở rộng theo n



Hình 10. Thời gian chạy theo n (log-log)

Thuật toán	n=4 (ms)	n=8 (ms)	n=12 (ms)	n=16 (ms)	n=20 (ms)	k (mọi n)	k (n ≥ 12)
Steepest asc. HC	0,1	1,3	6,5	19,1	59,6	3,92	4,32
Efficient Stochastic HC	0,4	4,8	16,1	37,0	85,4	3,31	3,25
Simulated Annealing	0,8	22,3	53,0	73,9	111,5	3,05	1,44

Bảng 13. Thời gian chạy trung bình (30 lượt mỗi n) và số mũ k ước lượng trong $t \sim n^k$

Thuật toán	Thành công n=4 (%)	Thành công n=8 (%)	Thành công n=12 (%)	Thành công n=16 (%)	Thành công n=20 (%)
Steepest asc. HC	43	3	7	3	3
Efficient Stochastic HC	37	7	7	0	0
Simulated Annealing	100	100	83	70	37

Bảng 14. Tỉ lệ thành công theo n (30 lượt mỗi n)

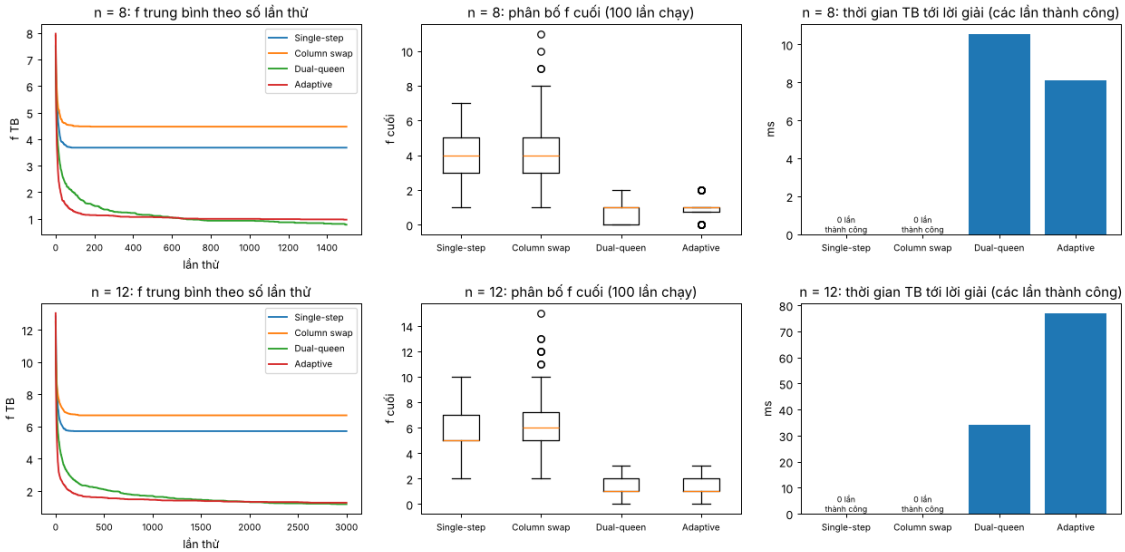
Độ dốc đường log-log cho số mũ thực nghiệm khoảng 3,9 (steepest-ascent), 3,3 (first-choice) và 3,1 (SA) khi tính trên mọi n. Giải thích: một lần gọi conflicts() tốn O(n); steepest-ascent mỗi bước quét n(n - 1) lân cận nên $O(n^3)$ mỗi bước, nhân với số bước tăng theo n, nên thực nghiệm gần n^4; first-choice mỗi lần thử chỉ tốn O(n) nhưng ngưỡng dừng $x \approx n^2 \ln n$ nên cỡ n^3 trở lên. Các số này chỉ là ước lượng từ năm điểm với tỉ lệ thành công rất khác nhau (leo đồi kết thúc sớm khi kẹt, còn SA chạy lâu hơn khi vẫn tìm nghiệm), nên phụ thuộc cài đặt và không nên coi là hệ số lý thuyết. Về chất lượng, SA giữ được tỉ lệ thành công cao hơn hẳn ở n ≥ 12 (xem Bảng 14, dù giảm dần theo n với ngân sách vòng lặp cố định), còn hai thuật toán leo đồi nhanh nhưng gần như luôn kẹt; vì vậy SA là lựa chọn mở rộng tốt nhất trong ba thuật toán, dù chi phí mỗi lượt cao hơn.

8.4. Bài nâng cao: các toán tử di chuyển cục bộ khác

Với first-choice hill climbing dùng cùng ngân sách thử cố định (1 500 lần với n = 8, 3 000 lần với n = 12), ta so sánh bốn toán tử: single-step (dịch một quân hậu lên/xuống một ô), column swap (hoán đổi hàng của hai cột), dual-queen (chuyển hai quân hậu sang hàng ngẫu nhiên cùng lúc) và adaptive (với xác suất 0,7 di chuyển quân hậu đang bị xung đột nhiều nhất, ngược lại chọn ngẫu nhiên một trong ba toán tử còn lại).

n	Toán tử	Thành công (%)	h cuối TB	Thời gian đến lời giải TB (ms)
8	Single-step	0	3,68	—
8	Column swap	0	4,47	—
8	Dual-queen	30	0,78	10,58
8	Adaptive	25	0,97	7,89
12	Single-step	0	5,71	—
12	Column swap	0	6,69	—
12	Dual-queen	12	1,18	32,16
12	Adaptive	10	1,27	75,32

Bảng 15. So sánh các toán tử di chuyển (100 lượt mỗi dòng)



Hình 11. Toán tử di chuyển: h trung bình theo số lần thử, phân bố h cuối và thời gian đến lời giải

Single-step và column swap không giải được lần nào. Single-step chỉ nhìn một lân cận rất nhỏ nên kẹt rất sớm. Column swap thì có lý do cấu trúc: hoán vị hai hàng giữ nguyên tập các hàng đã dùng, nên nó chỉ khám phá không gian hoán vị và không thể sửa các xung đột cùng hàng đã có sẵn từ bàn cờ ngẫu nhiên ban đầu; chỉ khi bàn cờ khởi tạo đã là hoán vị mới có cơ hội. Dual-queen (30% và 12%) và adaptive (25% và 10%) giải được một phần vì có bước nhảy lớn hơn; adaptive đến lời giải nhanh hơn ở n = 8 nhưng không vượt dual-queen ở n = 12. Cần nhớ rằng đây là first-choice với ngân sách cố định, các tỉ lệ thấp hơn nhiều so với SA hay restart.

9. Bảng ghi kết quả thí nghiệm (bảng 4.2)

Bảng dưới tổng hợp các thiết lập theo mẫu của đề. Cột "Xung đột / COUNT" là: COUNT trung bình (backtracking), số bước trung bình (Min-Conflicts) hoặc h cuối trung bình (n quân hậu); các giá trị này không cùng thang đo và không so sánh trực tiếp giữa các bài toán. Số màu / lượt là số màu của bài tô màu hoặc số lượt chạy của n quân hậu. Với tô màu, các dòng có 20 đồ thị (n = 100, 200), 100 đồ thị (n = 10) hoặc 20 đồ thị n = 10 có nghiệm (dòng "chỉ đồ thị có nghiệm"); thành công của backtracking nghĩa là tìm được nghiệm trước khi chạm trần 200 000 nút.

Bài toán	Kích thước	Thuật toán	Màu / lượt	Thành công (%)	Xung đột / COUNT	Thời gian (s)
Tô màu	n = 10	Backtracking	4	100	11,6	0,00002
Tô màu	n = 10	MRV + Backtracking	4	100	11,3	0,00006
Tô màu	n = 10	Forward Checking	4	100	11,2	0,00002
Tô màu	n = 10	MRV + Forward Checking	4	100	11,2	0,00005
Tô màu	n = 10	MRV + LCV + Forward Checking	4	100	11,2	0,00012
Tô màu	n = 10	Backtracking	3	5	47,3	0,00014
Tô màu	n = 10	Forward Checking	3	5	29,9	0,00009
Tô màu	n = 10	MRV + Forward Checking	3	5	26,3	0,00022
Tô màu	n = 100	Backtracking	4	100	1649,3	0,00597
Tô màu	n = 100	MRV + Backtracking	4	100	114,5	0,00391
Tô màu	n = 100	Forward Checking	4	100	517,3	0,00341
Tô màu	n = 100	MRV + Forward Checking	4	100	113,8	0,00378
Tô màu	n = 100	MRV + LCV + Forward Checking	4	100	114,0	0,00457
Tô màu	n = 200	Backtracking	4	90	37026,7	0,24341
Tô màu	n = 200	MRV + Backtracking	4	100	496,3	0,03998
Tô màu	n = 200	Forward Checking	4	95	12951,8	0,06336
Tô màu	n = 200	MRV + Forward Checking	4	100	443,8	0,03427
Tô màu	n = 200	MRV + LCV + Forward Checking	4	100	241,8	0,01647
Tô màu	n = 10	Min-Conflicts	4	100	12,1	0,00017
Tô màu	n = 20	Min-Conflicts	4	100	49,4	0,00107
Tô màu	n = 50	Min-Conflicts	4	100	374,5	0,01387
Tô màu	n = 100	Min-Conflicts	4	100	1338,3	0,09585
Tô màu	n = 200	Min-Conflicts	4	75	9591,5	1,15021
Tô màu	n = 10	Min-Conflicts	3	5	950,4	0,01151
Tô màu	n = 10	Min-Conflicts (chỉ đồ thị có nghiệm)	3	100	25,4	0,00026
n quân hậu	n = 4	Steepest asc. HC	100 lượt	47	0,57	0,00009
n quân hậu	n = 4	Ineff. Stochastic HC	100 lượt	29	0,89	0,00011
n quân hậu	n = 4	Efficient Stochastic HC	100 lượt	36	0,70	0,00042
n quân hậu	n = 4	Simulated Annealing	100 lượt	100	0,00	0,00097
n quân hậu	n = 4	Steepest asc. + Restart	100 lượt	100	0,00	0,00027
n quân hậu	n = 8	Steepest asc. HC	100 lượt	19	1,20	0,00140

Bài toán	Kích thước	Thuật toán	Màu / lượt	Thành công (%)	Xung đột / COUNT	Thời gian (s)
n quân hậu	n = 8	Ineff. Stochastic HC	100 lượt	11	1,34	0,00221
n quân hậu	n = 8	Efficient Stochastic HC	100 lượt	8	1,44	0,00476
n quân hậu	n = 8	Simulated Annealing	100 lượt	99	0,01	0,02239
n quân hậu	n = 8	Steepest asc. + Restart	100 lượt	100	0,00	0,01220

Bảng 16. Bảng ghi kết quả thí nghiệm (mẫu 4.2)

10. Nhận xét tổng hợp

Tính hợp lệ của nghiệm. Mọi nghiệm tô màu được kiểm tra lại bằng complete() và consistent() bằng assert trong mã thực nghiệm; mọi bàn cờ được báo cáo là thành công đều có h = 0 theo hàm conflicts(), vốn đã được đối chiếu với đếm vét cạn theo định nghĩa (Bài 5). Không có nghiệm nào được báo cáo mà chưa kiểm chứng.

Hiệu quả thuật toán. Với tô màu, backtracking với MRV + LCV + Forward Checking là lựa chọn tốt nhất ở n lớn (khoảng 242 nút và 15-20 ms ở n = 200) và còn có đảm bảo kết luận. Heuristic có chi phí mỗi nút cao hơn nên chỉ có lợi khi n đủ lớn. Min-Conflicts nhẹ nhưng chậm hơn và mất độ tin cậy ở n = 200. Với n quân hậu, leo đồi đơn lẻ rẻ nhưng chỉ thành công 8-19% ở n = 8; random restart và SA đều đưa tỉ lệ lên 99-100%, restart rẻ hơn ở cỡ nhỏ còn SA mở rộng tốt hơn ở n lớn.

Các trường hợp thất bại. (1) Tô màu 3 màu: đa số đồ thị vô nghiệm nên "thất bại" là đúng; backtracking chứng minh được, Min-Conflicts thì không. (2) Backtracking cơ bản hết giới hạn ở 10% đồ thị n = 200 và Forward Checking đơn thuần ở 5%. (3) Min-Conflicts chỉ giải 75% ở n = 200 với ngân sách đã chọn. (4) Leo đồi n quân hậu kẹt ở plateau. (5) Column swap và single-step không giải được n quân hậu trong ngân sách cố định.

Hạn chế của đánh giá thực nghiệm. Số đồ thị mỗi cỡ n còn ít (20), mỗi thiết lập chỉ dùng một chuỗi seed; độ lệch chuẩn lớn của backtracking cơ bản (mục 4.6) cho thấy trung bình có thể dao động. Thời gian chạy phụ thuộc máy và cài đặt Python (thuần Python, không tối ưu), COUNT và số lần đánh giá không cùng thang đo giữa các bài toán, trần 200 000 nút và các tham số của Min-Conflicts, SA (ngân sách bước, T_0, lịch) được chọn thủ công và ảnh hưởng đến kết quả. Đồ thị Delaunay có cấu trúc đặc biệt (3 màu hiếm khi tô được) nên kết luận không nên suy rộng sang mọi đồ thị. Các số mũ độ phức tạp ước lượng từ năm điểm n chỉ mang tính tham khảo.

11. Trả lời câu hỏi thảo luận

Câu 1. Khác nhau giữa complete() và consistent()? Một phép gán đầy đủ có luôn là nghiệm CSP không?

Complete() kiểm tra mọi biến đã được gán; consistent() kiểm tra không có ràng buộc nào bị vi phạm giữa các biến đã gán. Chúng độc lập: phép gán rỗng là consistent nhưng không complete; phép gán tô mọi đỉnh cùng màu là complete nhưng không

consistent (ca T4 ở Bảng 2). Vì vậy một phép gán đầy đủ không luôn là nghiệm; chỉ phép gán vừa đầy đủ vừa nhất quán mới là nghiệm.

Câu 2. MRV và Forward Checking tác động thế nào đến cây backtracking? Khi nào kiểm tra miền làm tăng chi phí?

MRV chọn biến ít giá trị hợp lệ nhất nên các nhánh thất bại bị phát hiện sớm, gần gốc cây; Forward Checking loại giá trị xung đột khỏi miền láng giềng và dừng ngay khi có miền rỗng, cắt bỏ các nhánh chắc chắn thất bại. Thực nghiệm ở n = 200 (Bảng 6): COUNT giảm từ khoảng 37 000 (backtracking cơ bản) xuống khoảng 444 (MRV + FC) và 242 (MRV + LCV + FC). Chi phí tăng khi n nhỏ hoặc bài toán dễ: mỗi nút phải tính lại số giá trị hợp lệ và duy trì/hoàn tác miền, nên ở $n \leq 50$ các cấu hình có heuristic chậm hơn Backtracking trơn dù COUNT thấp hơn (Bảng 6, n = 10 và 50).

Câu 3. Vì sao tô 3 màu có thể thất bại dù thuật toán đúng? Làm sao phân biệt hết giới hạn và chứng minh vô nghiệm?

Vì chính bài toán có thể vô nghiệm: đồ thị Delaunay là tam giác hóa phẳng, chỉ 3-tô được khi mọi đỉnh trong có bậc chẵn; thực nghiệm chỉ 5% đồ thị n = 10 có nghiệm 3 màu. Một thuật toán đúng không thể tìm ra nghiệm không tồn tại. Để phân biệt, solve_basic/solve_fc trả về trạng thái: "unsat" khi backtracking duyệt hết không gian mà không tìm được nghiệm (kết luận chắc chắn, 95% ở n = 10, 3 màu) còn "limit" khi vượt trần số nút (chưa kết luận). Min-Conflicts không phân biệt được hai trường hợp vì chỉ dừng khi hết ngân sách.

Câu 4. Hill climbing gặp cực trị cục bộ hoặc plateau thế nào? Random restart và SA xử lý ra sao?

Hill climbing chỉ nhận nước cải thiện chặt, nên dừng khi mọi lân cận đều tệ hơn (cực trị cục bộ) hoặc bằng (plateau). Trên n = 8 chỉ 11-14% lượt chạy đạt h = 0 và hầu hết trạng thái kẹt là plateau (Bảng 9). Random restart xử lý bằng cách chạy lại từ bàn cờ mới: xác suất tìm được nghiệm sau k lần là $1 - (1 - p)^k$ nên đạt 100% sau trung bình 7-9 lần khởi động (Bảng 10). SA chấp nhận cả nước tệ hơn với xác suất $\exp(-\Delta h/T)$ nên thoát được cực trị/plateau; chỉ 1% lượt chạy SA kết thúc ở cực trị cục bộ.

Câu 5. So sánh ba biến thể về số lần đánh giá láng giềng.

Steepest-ascent và Inefficient Stochastic đều đánh giá toàn bộ n(n - 1) láng giềng ở mỗi bước (56 với n = 8), khác nhau chỉ ở việc chọn nước tốt nhất hay chọn ngẫu nhiên trong các nước cải thiện (thực đo trung bình 232 và 334 lần đánh giá, nhiều hơn vì stochastic cần nhiều bước hơn: 5,08 so với 3,33). Efficient Stochastic (first-choice) chỉ đánh giá một láng giềng mỗi lần thử, nên mỗi bước rẻ hơn nhiều; tuy nhiên tổng số lần đánh giá vẫn cao (374 ở n = 8) vì phải chờ x = 338 lần thử liên tiếp không cải thiện mới kết luận kẹt. Lợi thế của first-choice rõ nhất khi số láng giềng rất lớn.

Câu 6. Vì sao cần nhiều lần chạy và cố định điều kiện thí nghiệm?

Các thuật toán ngẫu nhiên (bàn cờ ban đầu, chọn nước đi, SA) cho kết quả khác nhau mỗi lần chạy, nên một lần chạy không đại diện: chẳng hạn trên n = 8 một lần chạy leo đồi có thể thành công hoặc kẹt, còn tỉ lệ thành công thật chỉ lộ ra sau 100 lượt (8-19%). Ngay cả backtracking cơ bản cũng biến thiên rất lớn theo đồ thị (độ lệch chuẩn của COUNT lớn hơn cả trung bình ở mục 4.6). Cố định seed, cùng tập đồ thị/bàn cờ,

cùng tiêu chí dừng và trần số nút bảo đảm các thuật toán được so sánh công bằng và kết quả lặp lại được; thay đổi nhiều seed cho biết độ ổn định.

12. Kết luận

Lab04 cho thấy cùng một bài toán ràng buộc có thể giải bằng hai họ phương pháp: tìm kiếm có hệ thống (backtracking với MRV, LCV, Forward Checking) cho đảm bảo đầy đủ và chứng minh được vô nghiệm, và tìm kiếm cục bộ (Min-Conflicts, hill climbing, random restart, SA) nhẹ về bộ nhớ nhưng không có đảm bảo. Heuristic chọn biến (MRV) là cải tiến quan trọng nhất với backtracking; với tìm kiếm cục bộ, khả năng thoát plateau và cực trị (restart, SA) quyết định tỉ lệ thành công. Số màu quyết định độ khó nhiều hơn kích thước bài toán: 3 màu hầu như vô nghiệm trên đồ thị Delaunay còn 4 màu luôn có nghiệm và dễ tìm.

Bảng phân công:

MSSV	Họ Tên	Phân công	Đóng góp
3124720026	Huỳnh Tấn Phi Hùng	Bài 1–3: mô hình hóa CSP tô màu đồ thị, kiểm thử complete()/consistent(), Backtracking, cài đặt MRV và LCV; viết mục 2–3 báo cáo; trả lời câu hỏi thảo luận 1 và 2	25%
3124720072	Đỗ Xuân Triệu	Bài 8: so sánh tổng hợp các thuật toán, biểu đồ hội tụ, khả năng mở rộng (log-log), bài nâng cao về toán tử di chuyển; tổng hợp bảng 4.2 và nhận xét chung; trả lời câu hỏi thảo luận 6; hoàn thiện báo cáo	25%
3124720050	Lê Tấn Phát	Bài 4: cài đặt Forward Checking và Min-Conflicts, thực nghiệm 3/4 màu với nhiều kích thước n, đánh giá độ ổn định theo seed; viết mục 4 báo cáo; trả lời câu hỏi thảo luận 3	25%
3124720053	Lê Ngọc Phúc	Bài 5–7: biểu diễn bài toán n quân hậu, cài đặt 3 biến thể Hill Climbing, Random Restart, Simulated Annealing và khảo sát lịch nhiệt độ; viết mục 5–7 báo cáo; trả lời câu hỏi thảo luận 4 và 5	25%